

파머완

# 파머완 2~3장(사이킷런, 평가)

## 2장. 사이킷런으로 시작하는 머신러닝

### 01. 사이킷런 소개와 특징

- 파이썬 머신러닝 라이브러리 중 가장 많이 사용되는 라이브러리
- 파이썬 기반의 머신러닝을 위한 가장 쉽고 효율적인 개발 라이브러리 제공

#### 특징

- 가장 파이썬스러운 API 제공
- 머신러닝을 위한 매우 다양한 알고리즘 제공
- 개발을 위한 편리한 프레임워크와 API
- 오랜 기간 실전 환경에서 검증된 라이브러리

#### 버전 확인 코드

```
import sklearn
print(sklearn.__version__)
```

최신 버전 사이킷런 설치 코드

```
pip install -U scikit-learn
```

02. 첫 번째 머신러닝 만들어 보기 - 붓꽃 품종 예측하기

붓꽃 데이터 세트로 붓꽃의 품종을 분류

데이터 세트

- **피쳐(Feature)** : Sepal length, Sepal width, Petal length, Petal width (예측에 사용하는 정보)
- **레이블(Label)** : 붓꽃 품종(Setosa, Versicolor, Virginica) (예측하고 싶은 정답)

지도학습

- 피쳐와 레이블 데이터로 모델 학습 → 별도의 테스트 데이터 세트에서 미지의 레이블 예측
- **학습 데이터 세트**: 학습을 위해 주어진 데이터 세트
- **테스트 데이터 세트**: 머신러닝 모델의 예측 성능을 평가하기 위해 별도로 주어진 데이터 세트

주요 모듈 및 클래스

| 모듈 이름                   | 설명                                       | 모듈 내 api 예시            |
|-------------------------|------------------------------------------|------------------------|
| sklearn.datasets        | 사이킷런에서 자체 제공하는 데이터 세트를 생성하는 모듈의 모임       | load_iris()            |
| sklearn.tree            | 트리 기반 ML 알고리즘 구현 클래스 모임                  | DecisionTreeClassifier |
| sklearn.model_selection | 학습/테스트 데이터 분리, 하이퍼 파라미터로 평가 하기 위한 모듈의 모임 | train_test_split()     |

cf. 하이퍼 파라미터: ML 알고리즘별로 최적의 학습을 위해 직접 입력하는 파라미터들. ML 알고리즘의 성능을 튜닝할 수 있다.

## 분류 예측 프로세스

1. 데이터 세트 분리 : 학습 데이터, 테스트 데이터 분리. **train\_test\_split()** 사용
2. 모델 학습 : 학습 데이터를 기반으로 ML 알고리즘을 적용해 모델 학습. **fit()** 사용
3. 예측 수행 : 학습된 ML 모델을 이용해 테스트 데이터의 분류 예측. **predict()** 사용
4. 평가 : 예측된 결과값과 테스트 데이터의 실제 결과값을 비교해 모델의 성능 평가. **accuracy\_score()** 사용

## 03. 사이킷런의 기반 프레임 워크 익히기

### Estimator 이해 및 fit(), predict() 메서드

#### 지도 학습 - 분류, 회귀

- **Estimator 클래스** : 지도 학습의 모든 알고리즘을 구현한 클래스, **fit()**, **predict()** 이용
  - **Classifier** : 지도 학습 중 분류 알고리즘을 구현한 클래스
  - **Regressor**: 지도 학습 중 회귀 알고리즘을 구현한 클래스
  - **fit()** : ML 모델 학습
  - **predict()** : 학습된 모델 예측

활용: evaluation 함수 등에서 Estimator를 인자로 받음 -> 함수 내에서 fit(), predict() 호출 -> 평가, 튜닝 수행

#### 비지도학습 - 차원 축소, 클러스터링, 피쳐 추출 등

- **fit()**, **transform()** 사용
  - **fit()**: 입력 데이터의 형태에 맞춰 데이터를 변환하기 위한 사전 구조를 맞추는 작업 수행
  - **transform()**: 이후 실제 작업 수행
  - cf. 사이킷런에서는 둘을 결합한 **fit\_transform()** 도 제공.

# 사이킷런의 주요 모듈

| 분류                   | 모듈명                                     | 설명                                              |
|----------------------|-----------------------------------------|-------------------------------------------------|
| 예제 데이터               | <code>sklearn.datasets</code>           | 사이킷런의 내장된 예제 데이터 세트                             |
| 피처 처리                | <code>sklearn.preprocessing</code>      | 데이터 전처리(인코딩, 정규화, 스케일링 등)                       |
|                      | <code>sklearn.feature_selection</code>  | 알고리즘에 큰 영향을 미치는 피처를 우선순위로 선택 작업 수행              |
|                      | <code>sklearn.feature_extraction</code> | 텍스트/이미지 데이터의 벡처화된 피터 추출                         |
| 피처 처리 & 차원 축소        | <code>sklearn.decomposition</code>      | 차원 축소 알고리즘 지원 (PCA, NMF 등)                      |
| 데이터 분리, 검증 & 파라미터 튜닝 | <code>sklearn.model_selection</code>    | 학습용/테스트용 분리, 그리드 서치로 최적 파라미터 추출                 |
| 평가                   | <code>sklearn.metrics</code>            | 다양한 성능 특정 방법 제공 (Accuracy, Precision, Recall 등) |
| ML 알고리즘              | <code>sklearn.ensemble</code>           | 앙상블 알고리즘 제공(랜덤 포레스트, 에이다 부스트 등)                 |
|                      | <code>sklearn.linear_model</code>       | 회귀 관련 알고리즘, SGD 관련 알고리즘 제공                      |
|                      | <code>sklearn.naive_bayes</code>        | 나이브 베이즈 알고리즘 제공                                 |
|                      | <code>sklearn.neighbors</code>          | 최근접 이웃 알고리즘 지원 (K-NN 등)                         |
|                      | <code>sklearn.svm</code>                | 서포트 벡터 머신 알고리즘 제공                               |
|                      | <code>sklearn.tree</code>               | 의사 결정 트리 알고리즘 제공                                |
|                      | <code>sklearn.cluster</code>            | 비지도 클러스터링 알고리즘 제공(K-평균, 계층형, DBSCAN 등)          |
| 유틸리티                 | <code>sklearn.pipeline</code>           | 변환과 학습, 예측 등을 묶어서 실행 가능한 유틸리티 제공                |

## 머신러닝 모델을 구축하는 주요 프로세스

1. 피처 처리(피처의 가공, 변경, 추출)
  2. ML 알고리즘 학습/예측 수행
  3. 모델 평가
- (1~3을 반복)

## 내장된 예제 데이터 세트 api

- datasets 모듈에 있는 여러 API(load\_boston, diabetes 등)를 호출 → 데이터 세트 생성

## 분류나 회귀 연습용 예제 데이터

모두 sklearn.datasets 모듈에서 호출. **from sklearn.datasets import (api)**

- **datasets.load\_boston()** : 회귀용도, 미국 보스턴의 집 피처들과 가격에 대한 데이터 세트
- **datasets.load\_diabetes()** : 회귀 용도, 당뇨 데이터 세트
- **datasets.load\_breast\_cancer()** : 분류 용도, 위스콘신 유방암 피처들과 악성 / 양성 레이블 데이터
- **datasets.load\_digits()** : 분류 용도, 0에서 9 까지 숫자의 이미지 픽셀 데이터 세트
- **datasets.load\_iris()** : 분류 용도, 붓꽃에 대한 피처를 가진 데이터 세트

## fetch 계열 데이터

: 크기가 커서 패키지에 저장되어 있지 않음. 인터넷에서 내려 받아 홈 디렉터리 아래의 scikit\_learn\_data 라는 서브 디렉터리에 저장 후 추후 불러들이는 데이터, 최초 사용 시 인터넷 연결 필요.

- **fetch\_convtype()** : 회귀분석용 토지 조사 자료
- **fetch\_20newsgroups()** : 뉴스 그룹 텍스트 자료

등등,,

## 표본 데이터 생성기 for 분류, 클러스터링

- **datasets.make\_classifications()** : 분류를 위한 데이터 세트를 무작위로 생성. -> 노이즈 효과를 위한 데이터.
- **datasets.make\_blobs()** : 클러스터링을 위한 데이터 세트를 군집 지정 개수에 따라 무작위로 생성.

더 많지만, 위 2개 정도로도 여러 사례에 사용 가능하다.

## 분류나 회귀를 위한 연습용 예제 데이터의 구성(딕셔너리 형태)

- 내장 데이터 세트들은 대부분 Bunch 클래스 형태.(≒딕셔너리)
- 내장데이터세트 API 호출 후, .나 [ ]을 이용해 key값을 지정하여 각각 꺼낼 수 있음.
  - ex. iris\_data = load\_iris()→ iris.target, iris.data, iris['target'], iris['data']

### key (=피쳐들의 데이터 값)

- **data** : 피쳐의 데이터 세트. 즉 실제값(X) [ndarray]
- **target** : 분류 시 레이블 값, 회귀 시 숫자 결괏값 데이터 세트(y) [ndarray]
- **target\_names** : 개별 레이블의 이름. target의 숫자가 무슨 이름인지 알려줌.[ndarray, list]
- **feature\_names** : 피쳐의 이름. 즉 data의 열 이름 [ndarray, list]
- **DESCAR** : 데이터 세트에 대한 설명과 각 피쳐의 설명 [str]

|               |                                                                                           |       |       |       |                                                                          |        |
|---------------|-------------------------------------------------------------------------------------------|-------|-------|-------|--------------------------------------------------------------------------|--------|
| feature_names | <div>sepal length (cm)    sepal width (cm)    petal length (cm)    petal width (cm)</div> |       |       |       | target_names<br>setosa, versicolor, virginica<br>(0    ,    1    ,    2) |        |
|               |                                                                                           |       |       |       |                                                                          |        |
| data          | 5.1                                                                                       | 3.5   | 1.4   | 0.2   | 0                                                                        | target |
|               | 4.9                                                                                       | 3.0   | 1.4   | 0.2   | 1                                                                        |        |
|               | .....                                                                                     | ..... | ..... | ..... | .....                                                                    |        |
|               | 4.6                                                                                       | 3.1   | 1.5   | 0.2   | 2                                                                        |        |
|               | 5.0                                                                                       | 3.6   | 1.4   | 0.2   | 0                                                                        |        |

load\_iris의 key 값에 따른 데이터값들

## 04. Model Selection 모듈 소개

### 학습/테스트 데이터 세트 분리 - train\_test\_split(피쳐 데이터 세트, 레이블 데이터 세트)

- sklearn.model\_selection 모듈의 api. `from sklearn.model_selection import train_test_split`
- 전체 데이터를 학습 데이터, 테스트 데이터 세트로 분리
- 반환값: 튜플 형태 (학습용데이터의 피쳐데이터세트, 테스트용 데이터의 피쳐데이터세트, 학습용데이터의 레이블데이터세트, 테스트용데이터의 레이블데이터세트)

### 파라미터

- **test\_size** : 전체 데이터에서 테스트 데이터 세트의 비율 결정 (디폴트 0.25, 즉 25%)
- **train\_size** : 전체 데이터에서 학습용 데이터 세트의 비율 결정 → test\_size 를 사용하기 때문에 잘 사용하지 않음.
- **shuffle** : 데이터 분리 전 데이터를 미리 섞을지 결정 (디폴트 True)
- **random\_state** : 동일한 학습/테스트용 데이터 세트를 생성하기 위해 주어지는 난수 값

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split

dt_clf = DecisionTreeClassifier() #모델생성
iris_data=load_iris() #데이터세트api 호출

#테스트용, 학습용 분리
X_train, X_test, y_train, y_test = train_test_split(iris_data.data,
                                                    iris_data.target,
                                                    test_size=0.3,
                                                    random_state=121)

#학습 : 모델.fit(학습용 데이터)
dt_clf.fit(X_train, y_train)

#예측 : 모델.predict(테스트데이터)
pred=dt_clf.predict(X_test)
```

```
#정확도 : accuracy_score(정답, 예측한값)
print('예측 정확도: {0:.4f}'.format(accuracy_score(y_test, pred)))
```

## 교차 검증

- 과적합 문제를 개선하기 위해 다양한 학습과 평가 수행.
  - 과적합(Overfitting) : 모델이 학습 데이터에만 과도하게 최적화되어, 실제 예측을 다른 데이터로 수행할 경우에 예측 성능이 과도하게 떨어지는 것.
- 대부분의 ML 모델 성능 평가는 **교차 검증 기반으로 1차 평가** → **테스트 데이터 세트로 최종 평가**
- 즉, 데이터 세트 = **학습 데이터 세트(학습용 + 검증용)** + **테스트 데이터 세트**

## K-폴드 교차 검증

- 가장 보편적으로 사용되는 교차검증
- K개의 데이터 폴드 세트를 만들어서 K번만큼 각 폴드 세트에 학습, 검증 평가 반복 수행
  - ex. K=5. 데이터를 5등분하고, 각 데이터를 한 번씩 돌아가면서 검증용으로 사용 (나머지 4개는 학습용). 5개의 정확도를 구하고 평균 냄.
- **sklearn.model\_selection 모듈의 api. from sklearn.model\_selection import KFold**
- **KFold 객체 생성 : KFold 객체 = KFold(n\_splits=5)**
- **KFold 객체. split(X) : 학습용 데이터 인덱스의 1차원 array, 검증용 데이터 인덱스의 1차원 array 를 튜플로 묶어서 반환.**

```
#필요한 모듈, 함수 import
#data(X), target(y) 불러오기
#모델 생성
#KFold 객체 생성
#정확도 5개 값을 빈 리스트 생성.
#KFold객체.split(X) 함수의 반환값인 인덱스를 X(data), y(target)에 적용해서
X의 train, test, y의 train, test 데이터 추출
```

```
#fit, predict 로 학습 및 예측
#accuracy(y의 test, 예측값)으로 정확도 측정 후, 리스트에 하나씩 담기
#리스트에 담은 정확도들의 평균 계산
```

## Stratified K 폴드

- 불균형한 분포도를 가진 레이블 데이터 집합을 위한 K폴드 방식.
- 원본 데이터 레이블 분포 고려 → 해당 분포와 동일하게 학습/검증 데이터 세트 분배.
  - ex. 전체 데이터 100개 중 A=90개, B=10개이면, 어떤 폴드에는 B가 아예 없을 수 있는 문제 발생. 이때 모든 폴드에 A 90%, B 10%로 들어갈 수 있게 해주는 것
- 분류에서의 교차검증: K폴드보다는 Stratified K폴드 사용 / 회귀 : Stratified K폴드 지원x
- Kfold와 사용 방법은 거의 비슷하나, **split() 메서드에 피쳐데이터세트(X), 레이블데이터세트(y) 둘다 넣어주어야 한다.**

```
#StratifiedKfold를 이용해 붓꽃데이터 교차검증
# 필요한 모듈&함수 import

iris=load_iris()
features=iris.data #X
label=iris.target #y
dt_clf = DecisionTreeClassifier(random_state=156) #모델 생성

skfold = StratifiedKFold(n_splits=3) #Kfold 객체생성
n_iter=0
cv_accuracy=[] #정확도 값을 빈 리스트 생성

# StratifiedKFold의 split() 호출시 반드시 레이블 데이터 세트(y)도 추가 입력 필요
for train_index, test_index in skfold.split(features, label):
    # split()으로 반환된 인덱스를 이용해 학습용, 검증용 테스트 데이터 추출
    X_train, X_test = features[train_index], features[test_index]
    y_train, y_test = label[train_index], label[test_index]

    #학습 및 예측
    dt_clf.fit(X_train, y_train)
    pred = dt_clf.predict(X_test)
```

```

# 반복 시마다 정확도 측정
n_iter += 1
accuracy = np.round(accuracy_score(y_test, pred), 4)
cv_accuracy.append(accuracy)

# 교차 검증별 정확도 및 평균 정확도 계산
print(np.round(cv_accuracy, 4))
print(np.round(np.mean(cv_accuracy), 4))

```

## 교차 검증을 보다 간편하게 - cross\_val\_score()

- sklearn.model\_selection의 api. from sklearn.model\_selection import cross\_val\_score
- 주요 파라미터
  - **estimator** : 생성한 모델 입력. Classifier(-> Stratified K폴드사용) 또는 Regressor(-> K폴드사용)
  - **X** : 피쳐 데이터 세트
  - **y** : 레이블 데이터 세트
  - **scoring** : 예측 성능 평가 지표
  - **cv** : 교차 검증 폴드수
- 반환 값: scoring 파라미터로 지정된 성능 지표 측정값을 ndarray 로 반환.

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score, cross_validate
from sklearn.datasets import load_iris

iris_data = load_iris()
dt_clf = DecisionTreeClassifier(random_state=156)

data = iris_data.data
label = iris_data.target

# 성능 지표는 정확도(accuracy), 교차 검증 세트는 3개
scores = cross_val_score(dt_clf, data, label, scoring='accuracy', cv=3)

```

```
print(np.round(scores, 4)) #교차 검증별 정확도
print(np.round(np.mean(scores), 4)) #평균 검증 정확도
```

## GridSearchCV - 교차검증, 최적 하이퍼 파라미터 튜닝을 한 번에

- 교차검증 기반으로, 하이퍼 파라미터의 최적값을 순차적으로 적용해보면서 최적의 파라미터 도출
- 전체 데이터 세트를 학습/테스트 세트로 자동 분할 → 그리드에 기술된 파라미터를 차례로 적용.
  - cv=3회, 6개 파라미터 조합이면, 3\*6=18회의 학습과 평가가 이루어짐.
- **sklearn.model\_selection**의 api. `from sklearn.model_selection import GridSearchCV`
  - **cv\_results\_**속성 : 전체적인 결과. 데이터프레임으로 만든다.
  - **best\_params\_**속성 : 최고 성능을 나타낸 하이퍼 파라미터값
  - **best\_score\_**속성 : 그때의 평가 결과값(scoring)
  - **best\_estimator\_** : (refit=True일때) 이미 학습된 estimator 객체.

## 주요 파라미터

- **estimator** : 생성한 모델(알고리즘객체) 입력.
- **param\_grid** : 딕셔너리(key: estimator의 하이퍼 파라미터, value: 적용할 하이퍼 파라미터값을 리스트로 만든것)
- **scoring** : 예측 성능 평가 방법 지정.
- **cv** : 교차 검증 폴드수 . 설정안하면 사용하는 estimator의 score() 메서드를 기본.
- **refit** : True(디폴트) 설정 시 가장 최적의 파라미터를 찾은 뒤, 입력된 estimator 객체를 해당 파라미터로 **재학습까지 시킴**.

```
#모듈, 함수 import

#데이터 로딩, 분리
iris_data = load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris_data.data,
iris_data.target,
test_size=0.2, random_state=121)
```

```

dtree = DecisionTreeClassifier() #estimator 생성

parameters = {'max_depth':[1, 2, 3], 'min_samples_split':[2, 3]} #파라미터
#디렉터리

import pandas as pd

grid_dtree = GridSearchCV(dtree, param_grid=parameters, cv=3,
refit=True) #객체 설정
grid_dtree.fit(X_train, y_train) #순차적으로 학습

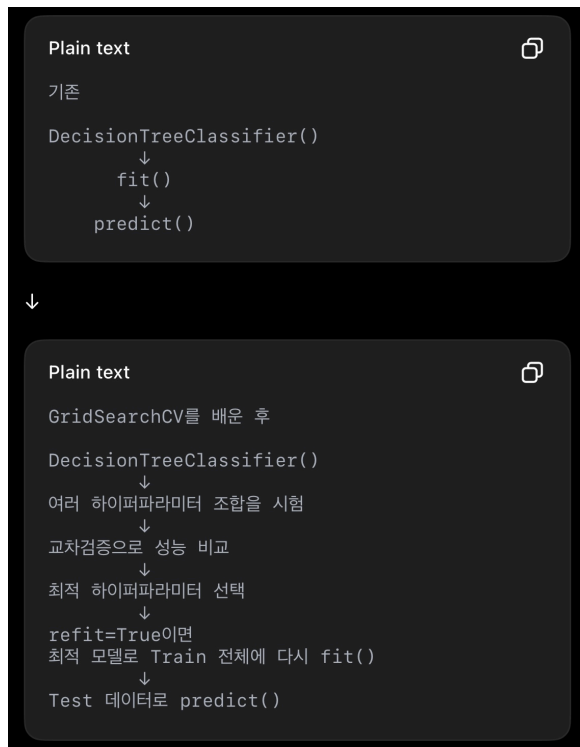
scores_df = pd.DataFrame(grid_dtree.cv_results_) #결과를 데이터프레임으로
만들기
scores_df[['params', 'mean_test_score', 'rank_test_score',
'split0_test_score', 'split1_test_score', 'split2_test_score']]

estimator = grid_dtree.best_estimator_ #최적의 파라미터로 학습된 estimator
반환
pred = estimator.predict(X_test) #predict로 테스트 (이미 학습은 됨-> fit
필요x)
print(accuracy_score(y_test, pred))

```

|   | params                                    | mean_test_score | rank_test_score | split0_test_score | split1_test_score | split2_test_score |
|---|-------------------------------------------|-----------------|-----------------|-------------------|-------------------|-------------------|
| 0 | { 'max_depth': 1, 'min_samples_split': 2} | 0.700000        | 5               | 0.700             | 0.7               | 0.70              |
| 1 | { 'max_depth': 1, 'min_samples_split': 3} | 0.700000        | 5               | 0.700             | 0.7               | 0.70              |
| 2 | { 'max_depth': 2, 'min_samples_split': 2} | 0.958333        | 3               | 0.925             | 1.0               | 0.95              |
| 3 | { 'max_depth': 2, 'min_samples_split': 3} | 0.958333        | 3               | 0.925             | 1.0               | 0.95              |
| 4 | { 'max_depth': 3, 'min_samples_split': 2} | 0.975000        | 1               | 0.975             | 1.0               | 0.95              |
| 5 | { 'max_depth': 3, 'min_samples_split': 3} | 0.975000        | 1               | 0.975             | 1.0               | 0.95              |

데이터프레임으로 만든 cv\_results\_ 속성



## 05. 데이터 전처리

- 데이터 전처리 모듈은 `preprocessing`. `from sklearn.preprocessing import (ap`  
i)

### 사이킷런의 ML 알고리즘 적용 전, 데이터 처리에 대한 기본 사항

#### 1. 결손값 허용 x

- 피쳐 값 중 Null 이 적으면 평균값 등으로 대체
- Null 값이 대부분이면 해당 피쳐 삭제 추천

#### 2. 문자열값을 입력값으로 허용 x

- 모든 문자열 값 숫자 형으로 변환 필요
- 카테고리형 피쳐 → 코드 값으로 변환, 텍스트형 피쳐 → 벡터화 or 불필요하면 삭제
- 식별자 피쳐는 예측 성능을 떨어뜨리므로 삭제 추천

## 데이터 인코딩 - 레이블 인코딩, 원-핫 인코딩

## 레이블 인코딩 (Label encoding)

- 카테고리 피처를 코드형 숫자값으로 변환하는 것 ex. 상품데이터에서 TV:1, 냉장고:2, 전자레인지:3.
- '01', '02' 같은 코드값 역시 문자열이므로 1,2 같은 숫자형 값으로 변환해야 한다.
- 숫자 값의 크고 작음이 순서, 중요도로 작동하면 안됨 → 선형 회귀 등에는 적용x, 트리 계열 알고리즘에는 적용 가능.
- `from sklearn.preprocessing import LabelEncoder`
- `LabelEncoder` 를 객체로 생성 → `fit()`, `transform()` 호출
  - 객체.`fit`(인코딩대상) \*인코딩대상: 원본리스트
  - 객체.`transform`(인코딩대상) : 인코딩 변환값(숫자) 반환
  - `classes_` 속성 : 인코딩 변환값에 대응하는 원본값 반환
  - 객체.`inverse_transform`(인코딩된 대상) : 디코딩 반환(인코딩 전으로 돌아가기)

## 원-핫 인코딩 (One-Hot Encoding)

- 피처값의 유형에 따라 새로운 피처를 추가 → 고유값에 해당하는 칼럼에만 1, 나머지칼럼에는 0 표시.

### 방법 1

- 입력값으로 2차원 데이터 필요. 인코딩대상= `np.array(items).reshape(-1,1)`
- 변환 후에는 (희소행렬 -> 밀집행렬) 변환 필요. 변환대상.`toarray()`
- `OneHotEncoder` 를 객체로 생성 → `fit()`, `transform()` 호출

### 방법2 : 판다스의 `get_dummies`(인코딩대상) 이용

| 원본 데이터 |           | 숫자로 인코딩 |           | 원-핫 인코딩 |     |    |     |       |     |           |
|--------|-----------|---------|-----------|---------|-----|----|-----|-------|-----|-----------|
| 상품 분류  | 가격        | 상품 분류   | 가격        | TV      | 냉장고 | 믹서 | 선풍기 | 전자레인지 | 컴퓨터 | 가격        |
| TV     | 1,000,000 | 0       | 1,000,000 | 1       | 0   | 0  | 0   | 0     | 0   | 1,000,000 |
| 냉장고    | 1,500,000 | 1       | 1,500,000 | 0       | 1   | 0  | 0   | 0     | 0   | 1,500,000 |
| 전자레인지  | 200,000   | 4       | 200,000   | 0       | 0   | 0  | 0   | 1     | 0   | 200,000   |
| 컴퓨터    | 800,000   | 5       | 800,000   | 0       | 0   | 0  | 0   | 0     | 1   | 800,000   |
| 선풍기    | 100,000   | 3       | 100,000   | 0       | 0   | 0  | 1   | 0     | 0   | 100,000   |
| 선풍기    | 100,000   | 3       | 100,000   | 0       | 0   | 0  | 1   | 0     | 0   | 100,000   |
| 믹서     | 50,000    | 2       | 50,000    | 0       | 0   | 1  | 0   | 0     | 0   | 50,000    |
| 믹서     | 50,000    | 2       | 50,000    | 0       | 0   | 1  | 0   | 0     | 0   | 50,000    |

# 피쳐 스케일링과 정규화

|                  |       |
|------------------|-------|
| sklearn          |       |
| └ preprocessing  | ← 모듈  |
| └ StandardScaler | ← 클래스 |
| └ MinMaxScaler   | ← 클래스 |
| └ Normalizer     | ← 클래스 |

- **피쳐 스케일링**: 서로 다른 변수의 값 범위를 일정한 수준으로 맞추는 작업. 대표적인 방법: 표준화, 정규화
  - **표준화**: 피쳐 각각이 평균 0, 분산 1 인 가우시안 정규 분포를 가진 값으로 변환. -> **사이킷런의 StandardScaler 클래스**.
  - **정규화**: 서로 다른 피쳐의 크기를 통일하기 위해 크기를 최소0~최대1의 값으로 변환. ex. 피쳐 A: 0~100, 피쳐 B: 0~100,000,000 를 모두 0~1로 단위통일. -> **사이킷런의 MinMaxScaler 클래스**

$$x_{i\_new} = \frac{x_i - \text{mean}(x)}{\text{stdev}(x)}$$

표준화

$$x_{i\_new} = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$

정규화

- **벡터 정규화**: 선형대수 개념의 정규화. -> **사이킷런의 Normalizer 클래스**.

$$x_{i\_new} = \frac{x_i}{\sqrt{x_i^2 + y_i^2 + z_i^2}}$$

벡터정규화

- \* StandardScaler / MinMaxScaler / Normalize 모두 입력값으로 2차원 데이터 필요.  
1차원이라면, np.array(데이터).**reshape(-1,1)**

## StandardScaler

- 표준화를 쉽게 지원하기 위한 클래스. 개별 피처를 평균0 분산1로 바꿔줌.
- RBF 커널을 이용하는 SVM, 선형 회귀, 로지스틱 회귀 전, 표준화 적용 필요.
- **from sklearn.preprocessing import StandardScaler**
  - 객체 = StandardScaler()
  - 객체.fit(데이터)
  - 객체.transform(데이터) : 스케일 변환된 데이터셋이 ndarray 로 반환. -> 데이터프레임 등 변환 필요.

## MinMaxScaler

- 정규화 클래스. 0과 1사이의 범위 값으로 바꿔줌.(음수 값이 있다면 -1~1로 변환)
- **from sklearn.preprocessing import MinMaxScaler**
  - 객체 = MinMaxScaler()
  - 객체.fit(데이터)
  - 객체.transform(데이터) : 스케일 변환된 데이터셋이 ndarray 로 반환. -> 데이터프레임 등 변환 필요.

## 학습 데이터와 테스트 데이터의 스케일링 변환 시 유의점

### StandardScaler, MinMaxScaler 객체 이용해 스케일링 변환 시

- fit() 메서드: 데이터 변환을 위한 기준 정보 설정. ex. 데이터 세트의 최댓값, 최솟값 설정.
- transform() 메서드: 위에서 설정된 정보를 이용해 데이터 변환.
- fit\_transform() : fit, transform 의 기능 한 번에 적용.

### 핵심원칙

#### : 테스트 데이터 세트에는 fit() 수행x

- 학습 데이터 세트에 fit(), transform() 적용 → 테스트 데이터 세트에는 학습 데이터 세트에 fit()을 적용한 결과를 이용해 transform() 변환.
- 즉, 학습 데이터로 fit()이 적용된 스케일링 기준 정보를, 그대로 테스트 데이터에 적용.

### 권장 방법

: 가능하다면 전체 데이터에 스케일링 적용 후, 학습/테스트 세트로 분리

```
from sklearn.preprocessing import MinMaxScaler
import numpy as np

# 학습 데이터는 0부터 10까지, 테스트 데이터는 0부터 5까지 값을 가지는 데이터
# 세트로 생성(2차원배열)
train_array = np.arange(0, 11).reshape(-1, 1)
test_array = np.arange(0, 6).reshape(-1, 1)

scaler = MinMaxScaler()
scaler.fit(train_array) # 학습용을 fit -> scaler 객체 안에 학습데이터기준이
# 저장됨
train_scaled = scaler.transform(train_array) # 학습용을 transform

test_scaled = scaler.transform(test_array) # 테스트용은 fit 호출안하고
# transform만.
```

## 3장. 평가

분류는 이진 분류, 멀티 분류로 나뉠 수 있다.

분류의 성능평가지표: 정확도, 오차행렬, 정밀도와 재현율, F1스코어, ROC AUC ← 이진  
분류에서 특히 중요한 지표들

### 01. 정확도(Accuracy)

$$\text{정확도(Accuracy)} = \frac{\text{예측 결과가 동일한 데이터 건수}}{\text{전체 예측 데이터 건수}}$$

: 실제 데이터에서 예측 데이터가 얼마나 같은지 판단하는 지표

- `from sklearn.metrics import accuracy_score`

## 한계점

- 이진 분류의 경우 데이터 구성에 따라 모델 성능 왜곡 가능.
- 특히 불균형한 분포에서 모델 성능을 판단할 때 부적합한 평가 지표.
  - ex. 100개의 데이터 중 90개 데이터 레이블이 0, 10개 데이터의 레이블이 1이라면, 무조건 0으로 예측 결과 반환하는 모델의 정확도는 무려 90%.
- 여러 분류 지표와 함께 적용 필요

## 02. 오차행렬

: 학습된 분류 모델이 예측을 수행하며 얼마나 헛갈리는지, 어떤 예측 오류가 발생하는지 함께 보여주는 지표

|                          |             | 예측 클래스<br>(Predicted Class)   |                               |
|--------------------------|-------------|-------------------------------|-------------------------------|
|                          |             | Negative(0)                   | Positive(1)                   |
| 실제 클래스<br>(Actual Class) | Negative(0) | <b>TN</b><br>(True Negative)  | <b>FP</b><br>(False Positive) |
|                          | Positive(1) | <b>FN</b><br>(False Negative) | <b>TP</b><br>(True Positive)  |

- **TN (True Negative)** : 예측값=실제값, 예측값이 0(부정)
  - **FP (False Positive)** : 예측값  $\neq$  실제값, 예측값이 1(긍정)
  - **FN (False Negative)** : 예측값  $\neq$  실제값, 예측값이 0(부정)
  - **TP (True Positive)** : 예측값=실제값, 예측값이 1(긍정)
- 
- `from sklearn.metrics import confusion_matrix`
  - `confusion_matrix(실제 답, 예측결과)` : 사분면 도표와 동일한 위치를 가진 ndarray 형태로 출력.(TN은 `array[0,0]`, FP는 `array[0,1]`, FN은 `array[1,0]`, TP는 `array[1,1]`에 해당)

- 불균형한 레이블 클래스를 가지는 이진 분류 모델에서는 중요하고 매우 적은 수의 결과값에 Positive를 설정해 1을 부과함. ex. 사기 행위 예측 모델: 사기 행위가 1, 정상행위가 0

## 정확도

: 예측 결과와 실제 값이 동일 / 전체 수 =  $(TN + TP) / (TN + FP + FN + TP)$

## 03. 정밀도와 재현율

: 불균형한 데이터 세트에서 정확도보다 더 선호되는 지표들. Positive 데이터 세트의 예측 성능에 좀 더 초점을 맞춘 평가 지표.

**정밀도(Precision) =  $TP / (FP + TP)$**

- 예측을 Positive 로 한 대상 중, 실제 값이 Positive 로 일치한 비율
- **TP를 높이는 데, FP을 낮추는 데 초점**
- =양성 예측도
- Negative(정상메일) 를 Positive(스팸메일) 로 잘못 판단하면 업무상 큰 영향이 발생하는 경우 중요. ex. 스팸메일 판단 모델
- `from sklearn.metrics import precision_score`

**재현율(Recall) =  $TP / (FN + TP)$**

- 실제 값이 Positive 인 대상 중, 예측과 실제 값이 Positive 로 일치한 비율
- **TP를 높이는 데, FN을 낮추는 데 초점**
- =민감도, TPR
- Positive(암) 를 Negative(정상) 로 잘못 판단하면 업무상 큰 영향이 발생하는 경우 중요. ex. 암 판단 모델, 금융 사기 적발 모델
- `from sklearn.metrics import recall_score`

## 정밀도/재현율 트레이드오프(Trade-off)

- 어느 한 쪽을 강제로 높이면 다른 쪽 수치는 떨어지기 쉬운 상호 보완적 관계
- 분류 결정 임계값을 조정하여 수치 조절 가능

- 임계값을 낮춤 → Positive 예측값이 더 많아짐. → 재현율은 증가, 정밀도는 감소

## cf. predict\_proba() 메서드

- predict() 메서드와 유사. 차이점은 반환 결과가 예측 결과 클래스값이 아닌, **예측 확률 결과**
- predict()는 predict\_proba() 메서드가 반환하는 확률 값을 가진 ndarray에서 정해진 임계값을 만족하는 ndarray의 칼럼 위치를 최종 예측 클래스로 결정.
  - **입력 파라미터**: 테스트 피쳐 데이터 세트(predict와 같음)
  - **반환 값**: 개별 클래스의 예측 확률을 ndarray m\*n 형태로 반환. 입력 테스트 데이터 세트의 표본 개수가 100개, 이진 분류라면, 100\*2 ndarray. 첫번째 칼럼은 0 (Negative), 두번째 칼럼은 1(Positive)

```
lr_clf.predict_proba(X_test)
# 로지스틱 회귀 모델 lr_clf에게 X_test(예측할 테스트 데이터)를 넣어서,
# 각 데이터가 각 클래스일 확률을 예측하기
```

|       | 0일 확률 | 1일 확률 |
|-------|-------|-------|
| 데이터 1 | 0.2   | 0.8   |
| 데이터 2 | 0.7   | 0.3   |
| 데이터 3 | 0.1   | 0.9   |

```
array([
  [0.2, 0.8],
  [0.7, 0.3],
  [0.1, 0.9]
])

lr_clf.predict_proba(X_test)[: , 1] : 1일 확률만 가져오는 것
```

## cf. Binarizer 클래스

- from sklearn.preprocessing import Binarizer
- 객체=Binarizer(threshold= )
- 객체.fit\_transform(2차원 ndarray): 지정된 threshold보다 같거나 작으면 0, 크면 1 반환.

## cf. precision\_recall\_curve() API

- 정밀도, 재현율의 관계를 계산해주는 함수.
- `from sklearn.metrics import precision_recall_curve`
- `precision, recall, thresholds = precision_recall_curve(y_test, pred_proba)`
  - 입력 파라미터: 실제 정답값(`y_test`), 1이라고 예측할 확률(`모델.predict_proba(X_test)[: , 1]`)
  - 반환 값: 임계값별 정밀도, 임계값별 재현율, 각각의 판단기준값(임계값) -> 각각 1차원 ndarray로 반환.

## 정밀도와 재현율의 맹점

- 정밀도를 100%로 만드려면 Positive 라고 예측하는 기준을 아주 엄격하게 하면 됨.
- 재현율을 100%로 만드려면, 모두 Positive라고 예측하면 됨.
- 정밀도, 재현율 중 하나만 스코어가 높고 하나는 스코어가 나쁜 분류는 성능이 좋지 않은 분류. 둘 사이의 균형이 중요.

## 04. F1 스코어

- 정밀도와 재현율을 결합한 지표.
- 정밀도와 재현율이 한쪽으로 치우치지 않을 때 상대적으로 높은 값.

$$F1 = \frac{2}{\frac{1}{recall} + \frac{1}{precision}} = 2 * \frac{precision * recall}{precision + recall}$$

- `from sklearn.metrics import f1_score`
- `f1_score(y_test, pred_prod)` : f1스코어 반환.

## 05. ROC 곡선과 AUC

### ROC 곡선 (Receiver Operation Characteristic Curve)

- **FPR(False Positive Rate)**이 변할 때 **TPR(True Positive Rate)**이 어떻게 변하는지를 나타내는 곡선
- X축: FPR, Y축: TPR(재현율)
- 곡선이 왼쪽 상단 모서리 쪽으로 가파르게 이동할수록(가운데 직선과 멀어질수록) 성능이 뛰어남.

## 사용되는 지표 : FPR, TPR

- TPR : 재현율(=민감도)
- TNR : 특이성 =  $TN / (FP+TN)$
- $FPR = FP/(FP+TN) = 1 - TNR$

## roc\_curve() API

- 예측 확률을 바탕으로 여러 임계값(threshold)을 자동으로 만들고, 각 임계값에서 FPR과 TPR을 계산해주는 함수.
- `precision_recall_curve()` API와 유사. 차이점은 반환값.
- `from sklearn.metrics import roc_curve`
- `fpr, tpr, thresholds = roc_curve(y_test, pred_proba)`
  - 입력 파라미터: 실제 정답값(`y_test`), 1이라고 예측할 확률(`모델.predict_proba(X_test)[: , 1]`)
  - 반환 값: FPR, TPR, 임계값 -> 각각 1차원 ndarray로 반환.
- 코드 결과를 보면, 임계값이 작아질수록 FPR이 점점 커진다. FPR이 조금씩 커질 때 TPR은 가파르게 커진다.

## AUC 스코어 (Area Under Curve)

- ROC 곡선 밑의 면적 값.
- 곡선이 왼쪽 상단 모서리 쪽으로 가파르게 이동할수록, AUC가 1에 가까워짐.(좋은 성능 수치)
  - 가운데 대각선 직선: 랜덤 수준의 이진 분류 AUC 값으로, 0.5
  - 보통의 분류는 0.5 이상의 AUC 값을 가짐.

# roc\_auc\_score() API

- AUC 값을 계산하는 함수.
- `from sklearn.metrics import roc_auc_score`
- `roc_auc_score( y_test, pred_proba )`: auc 값 반환
  - 입력 파라미터: 실제 정답값(`y_test`), 1이라고 예측할 확률(`모델.predict_proba(X_test)[:, 1]`)

| 파머완 카테고리의 다른 글     |            |
|--------------------|------------|
| 파머완 - 1장(넙파이, 판다스) | 2026.09.12 |

'파머완'의 다른글

이전글    파머완 - 1장(넙파이, 판다스)

현재글    : 파머완 2~3장(사이킷런, 평가)

관련글